Proprietà di HTML

Programmazione

HTML (HyperText Markup Language) è il linguaggio di base usato per creare la struttura e il contenuto di una pagina web.

Funziona attraverso tag che definiscono elementi come paragrafi, titoli, immagini e collegamenti.

Ogni pagina HTML parte con una struttura gerarchica che comprende il tag <html>, al cui interno troviamo <head> (informazioni e risorse della pagina) e <body> (contenuto visibile all'utente).

- **Linguaggio di markup**

HTML (HyperText Markup Language) non è un linguaggio di programmazione, ma un linguaggio di markup:
serve a descrivere la struttura e il significato dei contenuti di una pagina web (titoli, paragrafi, immagini, link), non a eseguire logica o calcoli.

- **Basato su tag**

HTML utilizza tag racchiusi tra < > per definire gli elementi della pagina.
Ogni tag rappresenta un contenuto o una funzione semantica (es. <p>, <h1>, <a>), spesso con tag di apertura e chiusura.

- **Struttura gerarchica (DOM)**

Gli elementi HTML sono organizzati in una struttura ad albero, detta DOM (Document Object Model).
Questa gerarchia permette al browser e a JavaScript di navigare, modificare e manipolare la pagina.

- **Semantica**

HTML moderno (HTML5) introduce tag semantici come <header>, <nav>, <article>, <section>, <footer>.
Questi migliorano:

  - **accessibilità**
  - **SEO**
  - **leggibilità del codice**

perché descrivono il significato del contenuto, non solo l'aspetto.

- **Indipendente dalla presentazione**

HTML si occupa solo del contenuto e della struttura, non dello stile grafico.
La presentazione è demandata ai CSS, mentre il comportamento dinamico è gestito da JavaScript (separazione dei ruoli).

- **Supporto agli ipertesti**

HTML nasce per il concetto di ipertesto:
grazie al tag <a> permette di collegare documenti, risorse e sezioni, creando la rete di pagine del Web.

- **Standardizzato**

HTML è uno standard definito dal W3C e dal WHATWG, garantendo:

  - **compatibilità tra browser**
  - **evoluzione controllata**
  - **retro-compatibilità con versioni precedenti**

- **Interpretato dal browser**

Il codice HTML viene interpretato direttamente dal browser, senza compilazione.
Il browser legge il file, costruisce il DOM e renderizza la pagina.

Contenteditable

Permette di rendere modificabile direttamente nel browser il contenuto di un elemento.

Uso tipico: editor di testo, prototipi veloci.

```
1. <div contenteditable="true">
2.   Puoi modificare questo testo direttamente!
3. </div>
```

Note:

- **Non salva automaticamente i dati**
- **Utile insieme a JavaScript per creare editor**

Data-* (custom data attributes)

Permette di salvare dati personalizzati sugli elementi HTML.

Uso tipico: passare informazioni a JavaScript.

HTML

```html
<button data-user-id="42">Profilo</button>
```

JS

```javascript
const btn = document.querySelector("button");
console.log(btn.dataset.userId); // "42"
```

Hidden

Nasconde un elemento senza usare CSS

HTML

```
1. <p hidden>Questo testo è nascosto</p>
```

Differenza da display:none :

- **È semantico (HTML puro)**
- **Può essere gestito via JavaScript**

draggable

Rende un elemento trascinabile (drag & drop).

HTML

```html
<img src="img.png" draggable="true">
```

Nota: funziona con eventi JavaScript come dragstart, drop.

Spellcheck

Attiva/disattiva il controllo ortografico.

HTML

```html
<textarea spellcheck="true"></textarea>
```

Valori:

- **true**
- **false**

Autocomplete

Controlla l’autocompletamento nei form.

HTML

```
1. <input type="email" autocomplete="email">
```

Valori utili:

- **name**
- **email**
- **username**
- **new-password**

Autofocus

Mette automaticamente il focus su un elemento al caricamento.

HTML

```
1. <input type="text" autofocus>
```

Attenzione:

- **Usarne solo uno per pagina**

required

Rende obbligatorio un campo nei form.

HTML

```
1. <input type="text" required>
```

Vantaggio: validazione senza JavaScript

Pattern

Permette di validare input con espressioni regolari.

HTML

```
1. <input type="text" pattern="[A-Za-z]{3}">
```

Esempio: solo 3 lettere

sandbox (iframe)

Limita i permessi di un iframe (sicurezza).

HTML

```
1. <iframe src="pagina.html" sandbox></iframe>
```

Permessi opzionali:

HTML

```
1. sandbox="allow-scripts allow-forms"
```

loading="lazy"

Carica immagini o iframe solo quando servono.

HTML

```html
<img src="foto.jpg" loading="lazy">
```

Vantaggio: performance migliori

srcset e sizes

Permettono immagini responsive.

HTML

```
1. <img
2.  src="small.jpg"
3.  srcset="small.jpg 500w, large.jpg 1000w"
4.  sizes="(max-width: 600px) 500px, 1000px">
```

tabindex

Controlla l’ordine di navigazione con TAB.

HTML

```html
<div tabindex="0">Focusabile</div>
```

Valori:

- **0 → naturale**
- **-1 → non raggiungibile via TAB**

aria-* (accessibilità)

Attributi per migliorare l’accessibilità.

HTML

```
1. <button aria-label="Chiudi">X</button>
```

Uso: screen reader, UI accessibile

Esempio completo:

```html
<!DOCTYPE html>
<html lang="it">
<head>
<meta charset="UTF-8">
<title>Demo attributi HTML</title>
<style>
  body { font-family: Arial; margin: 20px; }
  .box { border: 1px solid #ccc; padding: 10px; margin-bottom: 15px; }
</style>
</head>
<body>

<!-- 1. contenteditable -->
<div class="box">
  <h3>Editor (contenteditable)</h3>
  <div contenteditable="true">
   Modifica liberamente questo testo.
  </div>
</div>

<!-- 2. data-* -->
<div class="box">
  <h3>Data attribute</h3>
  <button data-user-id="42" onclick="mostraId(this)">
   Mostra ID utente
  </button>
</div>

<!-- 3. hidden -->
<div class="box">
  <h3>Hidden</h3>
  <p id="testoNascosto" hidden>Testo inizialmente nascosto</p>
  <button onclick="toggleHidden()">Mostra/Nascondi</button>
</div>

<!-- 4. draggable -->
<div class="box">
  <h3>Draggable</h3>
  <p draggable="true" ondragstart="dragStart(event)">
   Trascinami
  </p>
  <div ondrop="drop(event)" ondragover="allowDrop(event)"
      style="height:50px; border:1px dashed black;">
   Area di drop
  </div>
</div>

<!-- 5. spellcheck -->
<div class="box">
  <h3>Spellcheck</h3>
  <textarea spellcheck="true" rows="3"></textarea>
</div>

<!-- 6. autocomplete + 8. required + 9. pattern -->
<div class="box">
  <h3>Form</h3>
  <form id="formDemo">
   <input type="text" placeholder="Nome" autocomplete="name" required>
   <input type="text" placeholder="Codice (3 lettere)" pattern="[A-Za-z]{3}">
   <button type="submit">Invia</button>
  </form>
</div>

<!-- 7. autofocus -->
<div class="box">
  <h3>Autofocus</h3>
  <input type="text" placeholder="Focus automatico" autofocus>
</div>

<!-- 10. sandbox + 14. aria -->
<div class="box">
  <h3>Iframe sandbox</h3>
  <iframe sandbox srcdoc="<p>Contenuto isolato</p>"></iframe>
</div>

<!-- 11. loading lazy -->
<div class="box">
  <h3>Lazy loading</h3>
  <img src="https://via.placeholder.com/300" loading="lazy">
</div>

<!-- 12. srcset -->
<div class="box">
  <h3>Immagine responsive</h3>
  <img
   src="https://via.placeholder.com/300"
   srcset="https://via.placeholder.com/300 300w,
          https://via.placeholder.com/600 600w"
   sizes="(max-width: 600px) 300px, 600px">
</div>

<!-- 13. tabindex -->
<div class="box">
  <h3>Tabindex</h3>
  <div tabindex="0">Elemento focusabile con TAB</div>
</div>

<!-- 14. aria -->
<div class="box">
  <h3>ARIA</h3>
  <button aria-label="Chiudi finestra">X</button>
</div>

<script>
function mostraId(btn) {
  alert("User ID: " + btn.dataset.userId);
}

function toggleHidden() {
  const el = document.getElementById("testoNascosto");
  el.hidden = !el.hidden;
}

function allowDrop(ev) {
  ev.preventDefault();
}

function dragStart(ev) {
  ev.dataTransfer.setData("text", ev.target.innerText);
}

function drop(ev) {
  ev.preventDefault();
  const data = ev.dataTransfer.getData("text");
  ev.target.innerText = data;
}
</script>

</body>
</html>
```

Buon Davante a tutti